

Guía del alumno

Conceptos, Python con un almacén

Análisis Avanzado de Datos con Python

Cómo usar esta guía

Esta guía acompaña al cuaderno Conceptos. Sigue sus ocho pasos en el mismo orden y con el mismo nombre, Paso 1 de 8, Paso 2 de 8 y así hasta el final. Cada paso trae el código y la respuesta tal como aparecen en Colab, y una explicación en palabras simples.

No hace falta saber nada de programación. Todos los ejemplos son de un almacén de barrio, sin nada de energía todavía. La energía llega en el Lab de bienvenida, que viene después.

Lo que vas a poder hacer al terminar

- Guardar un dato en una caja con etiqueta y volver a pedirlo.
- Distinguir un texto de un número.
- Sacar una cuenta con datos que ya están guardados.
- Saber qué pasa cuando cambia un dato y qué hay que volver a correr.
- Guardar varios valores juntos en una lista.
- Leer una decisión con if y una repetición con for.
- Reconocer una tabla y sus filas y columnas.

Todas las respuestas de esta guía vienen de una corrida del cuaderno. Los precios del almacén son inventados para el ejemplo.

El cuaderno en una imagen

Todo este cuaderno pasa en un almacén de barrio.

El almacenero tiene letreros con precios, una boleta con palabras y cifras, una libreta con la lista de productos y una regla para cuando se acaba algo. Cada una de esas cosas es una idea de Python.

El letrero con el precio es una **variable**. La boleta distingue **texto** de **número**. La cuenta del total es un **cálculo**. La libreta con todos los precios es una **lista**. La regla de si no hay pan, galletas, es un **if**. Pasar los productos uno por uno por la caja es un **for**. Y la libreta con columnas es una **tabla**.

Python no inventó nada de esto. Lo hace igual que el almacenero, solo que sin cansarse.

Paso 1 de 8 · ¿Qué es una variable?

En el almacén, el precio del pan está escrito en un letrero. Cuando alguien pregunta cuánto cuesta, nadie vuelve a calcularlo, basta mirar el letrero. Una **variable** es eso mismo, una caja con una etiqueta donde se guarda un dato para usarlo después.

Paso 1 de 8

```
# Paso 1
# Una variable es una caja con una etiqueta. Guardamos el precio del pan.
precio_pan = 1500
print(precio_pan)
```

```
1500
```

La línea `precio_pan = 1500` se lee de derecha a izquierda. Primero está el dato, **1500**, y después la etiqueta con la que se va a pedir, `precio_pan`. El signo igual no quiere decir que dos cosas sean iguales, quiere decir *guarda esto en la caja que se llama así*.

La segunda línea, `print`, muestra en pantalla lo que hay dentro de la caja. Por eso debajo de la celda aparece el número 1500. Las líneas que empiezan con el signo `#` son comentarios. Python no las lee, están ahí para que una persona entienda el código.

Paso 2 de 8 · ¿Texto o número?

En la boleta del almacén hay palabras y hay cifras. Las palabras dicen qué se compró y las cifras dicen cuánto. Python también distingue esas dos cosas, y lo hace por una señal muy simple, las comillas.

Paso 2 de 8

```
# Paso 2
# Lo que va entre comillas es texto. Lo que no lleva comillas es número.
producto = "pan"
cantidad = 3
print(producto, cantidad)
```

```
pan 3
```

"pan" va entre comillas, así que es **texto**. 3 va sin comillas, así que es un **número**. En pantalla se ven parecidos, pero no se comportan igual. Con un número se puede sacar una cuenta. Con un texto no. El `print` de esta celda muestra dos cajas a la vez, separadas por una coma. Python las escribe una al lado de la otra con un espacio entre medio.

Paso 3 de 8 · ¿Se puede calcular con las cajas?

La cajera no anota el total de memoria. Toma el precio del pan, lo multiplica por la cantidad y anota el resultado. Python hace lo mismo con lo que ya está guardado.

Paso 3 de 8

```
# Paso 3
# Con los números guardados se puede calcular. El asterisco es multiplicar.
total = precio_pan * cantidad
print("Total a pagar", total)
```

```
Total a pagar 4500
```

`precio_pan * cantidad` usa las dos cajas de los pasos anteriores. El asterisco es multiplicar. El resultado se guarda en una caja nueva, `total`, y la pantalla muestra 4500, que es 1500 por 3.

La suma se escribe con `+`, la resta con `-` y la división con `/`. Son los mismos signos de una calculadora, salvo el de multiplicar, que es el asterisco.

Paso 4 de 8 · ¿Qué pasa si cambio lo que hay en una caja?

El pan subió. El almacenero borra el letrero y escribe el precio nuevo. Pero la boleta que ya entregó no cambia sola, sigue diciendo lo que decía cuando se imprimió.

Paso 4 de 8

```
# Paso 4
# Guardar otro valor con la misma etiqueta reemplaza el anterior.
precio_pan = 1700
print("Precio nuevo del pan", precio_pan)
print("Total", total)
```

```
Precio nuevo del pan 1700
Total 4500
```

Guardar otro valor con la misma etiqueta reemplaza el anterior. Ahora `precio_pan` vale 1700. Pero `total` sigue en 4500, porque se calculó en el paso 3, cuando el pan costaba 1500, y nadie le pidió a Python que lo volviera a calcular.

Esta es la idea más importante del cuaderno. Python no recalcula solo. Si cambias un dato y quieres el resultado nuevo, tienes que volver a correr la celda que hace la cuenta. Para ver el total con el precio nuevo, basta volver a correr el paso 3.

Paso 5 de 8 · ¿Cómo guardo varios valores juntos?

En el almacén no hay un solo producto. Tener una caja con etiqueta para cada precio sería un enredo. Lo práctico es una sola lista con todos los precios, en orden.

Paso 5 de 8

```
# Paso 5
# Una lista va entre corchetes, con los valores separados por comas.
precios = [1500, 990, 2300]
print(precios)
print("Cantidad de productos", len(precios))
print("Suma de los precios", sum(precios))
```

```
[1500, 990, 2300]
Cantidad de productos 3
Suma de los precios 4790
```

Una **lista** va entre corchetes, con los valores separados por comas. Toda la lista queda guardada bajo una sola etiqueta, `precios`.

Con la lista completa se pueden hacer preguntas de una vez. `len` cuenta cuántos valores tiene, que acá son 3. `sum` los suma, y da 4790. Si mañana el almacén vende diez productos, esas mismas dos líneas siguen sirviendo sin cambiar nada.

Paso 6 de 8 · ¿Cómo decide Python?

Al llegar al almacén uno decide según lo que hay. Si hay pan, compra pan. Si no hay, compra galletas. Python decide igual, con una pregunta que se responde sí o no.

Paso 6 de 8

```
# Paso 6
# if pregunta algo. Si la respuesta es sí, hace lo de abajo.
# Si no, hace lo del else.
stock_pan = 0 #@param {type:"slider", min:0, max:10, step:1}

if stock_pan > 0:
    print("Hay pan. Compro pan.")
else:
    print("No hay pan. Compro galletas.")
```

No hay pan. Compro galletas.

El deslizador de la derecha cambia el valor de `stock_pan`. El cuaderno parte en 0, así que la pregunta `stock_pan > 0`, que se lee *el stock es mayor que cero*, se responde que no, y Python hace lo que está bajo el `else`.

Prueba mover el deslizador a cualquier número mayor que cero y vuelve a correr la celda. La respuesta cambia a Hay pan. Compro pan. Mover el deslizador no basta, hay que volver a correr la celda para que Python vuelva a decidir.

Las líneas que van corridas hacia la derecha son las que dependen de cada respuesta. Ese espacio al comienzo de la línea no es adorno, es lo que le dice a Python qué hacer en cada caso.

Paso 7 de 8 · ¿Cómo repite Python una tarea?

En la caja, la cajera pasa los productos uno por uno. Hace lo mismo con cada uno hasta que se acaban. Python repite una tarea de la misma forma.

Paso 7 de 8

```
# Paso 7
# for repite lo de abajo una vez por cada valor de la lista.
for precio in precios:
    print("Pasa por la caja un producto de", precio)
```

Pasa por la caja un producto de 1500
Pasa por la caja un producto de 990
Pasa por la caja un producto de 2300

`for precio in precios` quiere decir *para cada precio de la lista*. En cada vuelta, la caja `precio` guarda un valor distinto de la lista, primero 1500, después 990 y al final 2300. La línea de abajo, la que va corrida a la derecha, se repite una vez por cada valor.

La lista tiene tres precios, así que la frase aparece tres veces. Si la lista tuviera cien, la celda no cambiaría y la frase aparecería cien veces.

Paso 8 de 8 · ¿Qué es una tabla?

La libreta del almacén tiene una columna con los productos y otra con los precios. Eso es una tabla, como una planilla. Python también arma tablas, y es la forma en que vamos a trabajar todo el curso.

Paso 8 de 8

```
# Paso 8
# Una tabla junta varias listas como columnas.
import pandas as pd

almacen = pd.DataFrame({
    "producto": ["pan", "leche", "queso"],
    "precio": precios,
})
almacen
```

	producto	precio
0	pan	1500
1	leche	990
2	queso	2300

La primera línea, `import pandas as pd`, trae la herramienta de Python que arma tablas. Se llama `pandas`, y se escribe una sola vez al principio.

La tabla junta dos listas como columnas. Una con los nombres de los productos y otra con la lista `precios` del paso 5. Cada fila es un producto. Los números de la izquierda, 0, 1 y 2, los pone `pandas` solo para numerar las filas, y empiezan en cero.

Para repasar

Referencia rápida

Lo que se escribe	Para qué sirve
<code>nombre = dato</code>	Guarda el dato en una caja con esa etiqueta
<code>"pan"</code>	Un texto, siempre entre comillas
<code>1500</code>	Un número, sin comillas
<code>print(x)</code>	Muestra en pantalla lo que hay en la caja
<code>+</code> <code>-</code> <code>*</code> <code>/</code>	Sumar, restar, multiplicar y dividir
<code>[1500, 990, 2300]</code>	Una lista con varios valores, en orden
<code>len(lista)</code> <code>sum(lista)</code>	Cuántos valores tiene y cuánto suman
<code>if</code> <code>else</code>	Hace una cosa si la respuesta es sí y otra si es no
<code>for x in lista:</code>	Repite lo de abajo una vez por cada valor
<code># al comienzo</code>	Un comentario, Python no lo lee

Si algo sale mal

Lo que aparece en pantalla	Qué pasó
<code>NameError: name 'precio_pan' is not defined</code>	Python no conoce esa etiqueta. Suele pasar si no corriste el paso donde se guarda, o si el nombre está mal escrito. Revisa la ortografía y corre los pasos en orden.
<code>SyntaxError: invalid syntax</code>	Falta una comilla, un paréntesis o un corchete de cierre. Mira la flecha que Python dibuja debajo de la línea.
<code>IndentationError</code>	Una línea que tenía que ir corrida a la derecha no lo está, o al revés. Pasa en los pasos 6 y 7.
El total no cambió después de cambiar un precio	No es un error. Python no recalcula solo. Vuelve a correr la celda que hace la cuenta.

Tres preguntas de repaso

1. Qué quiere decir el signo igual en Python

Que el dato de la derecha se guarda en la caja que tiene la etiqueta de la izquierda. No quiere decir que dos cosas sean iguales.

2. Por qué el total del paso 4 siguió en 4500

Porque se calculó antes de cambiar el precio y nadie volvió a correr la cuenta. Python no recalcula solo.

3. Cuántas veces se repite la línea de abajo de un for

Una vez por cada valor de la lista. Con tres precios, tres veces.

Python con un almacén en una plana

Para imprimir y tener al lado durante el curso

Las ocho ideas

Idea	Se escribe	En el almacén
Variable	<code>precio_pan = 1500</code>	El letrero con el precio
Texto y número	<code>"pan" · 3</code>	Las palabras y las cifras de la boleta
Cálculo	<code>precio_pan * cantidad</code>	La cuenta del total
Reemplazo	<code>precio_pan = 1700</code>	El letrero nuevo cuando sube el pan
Lista	<code>[1500, 990, 2300]</code>	La libreta con todos los precios
Decisión	<code>if · else</code>	Si no hay pan, galletas
Repetición	<code>for precio in precios:</code>	Pasar los productos por la caja
Tabla	<code>pd.DataFrame(...)</code>	La libreta con columnas

Las tres cosas que más confunden al principio

El signo igual guarda	No compara, guarda el dato en la caja
Python no recalcula solo	Si cambias un dato, vuelve a correr la cuenta
Las celdas van en orden	Una celda usa lo que guardaron las de arriba

Lo que sigue es el Lab de bienvenida. Ahí estas mismas ideas se usan con las centrales de energía del curso.